

Documentación Técnica — TFG D&D Manager

Gestor fullstack de campañas y personajes D&D 5e.

Stack: FastAPI · MongoDB · Angular 21 · TypeScript

Tabla de Contenidos

1. [Arquitectura General](#)
2. [Base de Datos](#)
3. [Backend — FastAPI](#)
4. [Frontend — Angular 21](#)
5. [Flujos End-to-End](#)

1. Arquitectura General

```
TFG_DND/  
├── API/                                # Backend FastAPI → puerto 8000  
│   ├── main.py  
│   ├── config.py  
│   ├── db.py  
│   ├── models/  
│   ├── routes/  
│   ├── services/  
│   └── utils/  
└── APP/                                # Frontend Angular 21 → puerto 4200  
    └── src/app/  
        ├── app.routes.ts  
        ├── auth.guard.ts  
        ├── interfaces/  
        ├── services/  
        ├── components/  
        └── pages/
```

Comunicación:

Browser → Angular SPA → FastAPI → MongoDB (Docker, red interna "mongodb:27017")

Auth: JWT sin expiración firmado con HS256. Se guarda en `localStorage`. Cada request protegido lo envía en `Authorization: Bearer <token>`. El backend lo decodifica con `python-jose`.

2. Base de Datos

Colecciones activas

Colección	Descripción
<code>users</code>	Usuarios del sistema
<code>characters</code>	Fichas de personaje
<code>monsters</code>	Monstruos (oficiales + comunidad)
<code>spells</code>	Hechizos
<code>backgrounds</code>	Trasfondos
<code>items</code>	Ítems de equipo (todos los tipos)
<code>racess</code>	Razas D&D
<code>subraces</code>	Sub-razas
<code>classes</code>	Clases. Incluye <code>levels[]</code> embebido con los 20 niveles.
<code>subclasses</code>	Subclases. Incluye <code>levels[]</code> embebido.
<code>traits</code>	Rasgos raciales
<code>features</code>	Features de clase
<code>class_progression_config</code>	Config de columnas de progresión por clase

Nota: Las colecciones `class_levels` y `subclass_levels` fueron eliminadas tras la migración. Los niveles viven ahora dentro de cada documento de `classes/subclasses` en el campo `levels[]`.

Campo `created_by`

Todos los documentos creados por usuarios tienen `created_by: <user_id>`. Los datos oficiales importados tienen `created_by: 'oficial'`. Esto permite el filtrado por fuente (Official / Community / Own) en el frontend.

Colección `class_progression_config`

Controla la tabla de progresión de clase que se muestra en `class-detail` y `character-detail`.

```
{
  "class_name": "barbarian",
  "hiddenKeys": [],
  "spellSlots": false,
  "progressionColumns": [
    {
      "id": "rages-count",
      "label": "Rages per Day",
      "source": "class_specific",
      "key": "rage_count",
      "progression": [
        { "level": 1, "value": "2" },
        { "level": 3, "value": "3" }
      ]
    }
  ]
}
```

```
}
]
}
```

- **hiddenKeys**: keys de `class_specific` a suprimir en la UI
- **spellSlots**: si `true`, se generan columnas de spell slots automáticamente
- **progression[]**: para calcular el valor, se toma el último entry cuyo `level ≤ nivelPersonaje`

3. Backend — FastAPI

`main.py` — Punto de entrada

Por qué existe: Es el bootstrap de FastAPI. Configura todo lo que aplica globalmente antes de que llegue cualquier request.

`app = FastAPI()`

Instancia la aplicación. Se hace aquí (nivel módulo) para que todas las importaciones posteriores trabajen sobre el mismo objeto.

Middleware CORS

Qué hace: Permite peticiones desde cualquier origen (`allow_origins=["*"]`).

Por qué está: En desarrollo, el frontend en `localhost:4200` y el backend en `localhost:8000` son orígenes distintos. Sin CORS el browser bloquea los requests.

Dónde se configura: En `main.py` con `app.add_middleware(CORSMiddleware, ...)`. No hay llamador — se aplica automáticamente a todas las rutas.

`trace_http(request, call_next)`

Qué hace: Intercepta cada HTTP request/response. Loguea método, path, status y tiempo de respuesta en ms. Añade `X-Request-ID` al response.

Por qué está: Permite trazar qué endpoints se llaman y cuánto tardan sin tocar el código de cada route.

Dónde se llama: FastAPI lo invoca automáticamente antes/después de cada handler por ser middleware `@app.middleware("http")`.

`startup() / shutdown()`

Qué hace: `startup` imprime log de arranque. `shutdown` llama a `close_db()` para cerrar la conexión con MongoDB limpiamente.

Por qué está: Sin el `shutdown`, el cliente Motor quedaba abierto al parar el servidor, generando warnings.

Dónde se llama: FastAPI los invoca automáticamente en el inicio/parada del servidor por `@app.on_event`.

`GET /` y `GET /health`

Qué hace: Endpoints de comprobación rápida. `/health` devuelve `{"status": "ok"}`.

Por qué están: Docker puede configurar un healthcheck sobre `/health` para saber si el contenedor está listo. `/` es el "está vivo" más básico.

Dónde se llaman: Desde Docker healthcheck y manualmente en desarrollo para verificar que el servidor arranca.

`config.py` — Configuración

Por qué existe: Centraliza todas las variables de entorno en un único lugar. Cualquier módulo que necesite una variable importa de aquí en vez de leer `os.environ` directamente — facilita el testing y evita typos.

Variables:

Variable	Tipo	Uso
<code>SECRET_KEY</code>	str	Clave para firmar/verificar JWT
<code>ALGORITHM</code>	str	Algoritmo JWT (<code>HS256</code>)
<code>ACCESS_TOKEN_EXPIRE_MINUTES</code>	int	Definido en <code>.env</code> pero el token NO expira (sin <code>exp</code> en payload)
<code>MONGODB_URI</code>	str	URI completa construida desde usuario/pass/host/port/db
<code>MONGODB_DATABASE</code>	str	Nombre de la BD en Mongo
<code>MONGODB_COLLECTION_*</code>	str	Nombres de colecciones como constantes

Dónde se importa: En `db.py` para conectarse, en `auth_service.py` para firmar JWT, en todos los services que necesitan el nombre de colección.

`db.py` — Conexión MongoDB

Por qué existe: Motor (driver async de MongoDB) requiere que el cliente sea un singleton — crear un cliente por request es muy costoso y provoca agotamiento de conexiones.

`get_db()` → `AsyncIOMotorDatabase`

Qué hace: Si `_client` es `None`, crea el `AsyncIOMotorClient`. Devuelve la BD de la config.

Por qué está: Patrón singleton lazy — el cliente se crea la primera vez que se necesita, no al importar el módulo.

Dónde se llama: En cada función de servicio que accede a MongoDB (`await get_db()`). Es la única forma de obtener la BD en toda la aplicación.

`close_db()`

Qué hace: Cierra el cliente Motor y pone `_client = None`.

Por qué está: Sin esto, Motor deja conexiones abiertas al parar el servidor.

Dónde se llama: Desde `shutdown()` en `main.py` vía el evento de apagado de FastAPI.

models/Generico.py — Tipos base

Por qué existe: Evita repetir los campos comunes (`id`, `created_by`, `created_at`, `updated_at`) en cada modelo. Todos los schemas de dominio heredan de aquí.

BaseSchema

Clase Pydantic base con `id`, `created_by`, `created_at`, `updated_at` opcionales.

Por qué: MongoDB genera `_id` (ObjectId) pero la API devuelve `id` (string). El campo `created_by` permite saber quién creó cada documento para el sistema de permisos.

Hereda: `BackgroundSchema`, `UserSchema` (indirectamente), todos los schemas de ítems.

ResourceReference

`{index, name, url}`. Representa una referencia a otra entidad D&D.

Por qué: En D&D 5e casi todo se referencia entre sí (una clase tiene proficiencias, una proficiencia tiene un índice, etc.). Este tipo unifica esa estructura de referencia.

Dónde se usa: En `Monster.ProficiencySchema`, `Background.EquipmentItemSchema`, `Character.ApiReferenceSchema`, y docenas de campos de la API D&D 5e.

CostSchema

`{quantity, unit}`. Coste de un ítem.

Por qué: La API D&D 5e devuelve el coste siempre como objeto con cantidad y unidad (`{quantity: 10, unit: "gp"}`). Tenerlo como schema evita duplicar la definición en cada tipo de ítem.

Dónde se usa: En todos los modelos de ítem (`Weapon`, `Armor`, `AdventuringGear`, etc.).

models/User.py

UserSchema

Campos: `username`, `email`, `password` (opcional — solo se incluye para verificar login), `role` (`"user"` | `"admin"`).

Por qué password es opcional: Al devolver usuarios desde `GET /users`, la contraseña se excluye. Solo se incluye cuando `include_password=True` en el service (al hacer login).

Dónde se usa: En `routes/users.py` como tipo de entrada/salida, en `services/users_service.py` para crear/actualizar.

models/Login.py

LoginRequest

`{email, username, password}`. Email y username son opcionales — se acepta cualquiera de los dos.

Por qué: Permite login con email O username sin tener dos endpoints separados.

LoginResponse

```
{access_token, token_type, isAdmin, user_id, name, email}.
```

Por qué: El frontend necesita saber: el token, si es admin (para mostrar/ocultar funciones) y el user_id (para el sistema de permisos en `canEdit`).

Dónde se usa: Como `response_model` en `POST /auth/login` y `POST /auth/token`.

VerifyTokenResponse

```
{valid: bool, user: dict}.
```

Por qué: El frontend llama a `/auth/verify` en cada página para obtener el `user_id` e `isAdmin` del token almacenado en `localStorage`. Sin este endpoint, no habría forma de obtener el `user_id` sin decodificar el JWT en el cliente.

Dónde se usa: Como `response_model` en `POST /auth/verify`.

models/Monster.py

Por qué existe: Valida que los datos de un monstruo cumplan la estructura D&D 5e antes de guardarlos. Usa `ConfigDict(extra="allow")` para no rechazar campos adicionales como `created_by` que no son parte del modelo oficial.

Sub-modelos relevantes:

- `SpeedSchema`: velocidades de movimiento (walk, fly, swim, climb, burrow)
- `AbilitySchema`: habilidades especiales, acciones, reacciones. Puede tener `spellcasting` anidado
- `ProficiencySchema`: `{value: int, proficiency: ResourceReference}`

Dónde se usa: En `monsters_service.create()` para validar antes de insertar, en `routes/monsters.py` no directamente (el parsing lo hace `parse_form_or_json`).

models/Spell.py

Por qué existe: Los hechizos tienen muchos campos opcionales (daño, DC de salvación, área de efecto) que no todos los hechizos tienen. Pydantic permite modelarlos como `Optional` con defaults `None`.

Campo `created_by`: Definido explícitamente en `Spell` (a diferencia de `Monster` que usa `extra="allow"`).

Por qué: Se necesita para el sistema de filtrado por fuente y para `canEdit` en el frontend.

models/Background.py

Por qué existe: Los trasfondos tienen estructura compleja (opciones de personalidad, ideales, vínculos, defectos) que puede venir en dos formatos: el propio (lista simple) o el de la API D&D 5e (objeto anidado con `from.options`). `extra="allow"` tolera ambos.

models/Character.py

Por qué existe: El personaje es el documento más complejo. El schema valida que tenga los campos obligatorios (nombre, clase, raza, background, stats) antes de guardarlo.

Campos clave:

- `character_class` con alias `class` (en MongoDB el campo se llama `class`, pero en Python `class` es palabra reservada)
- `inventory.items`: lista de `InventoryItemSchema` con tipo, estado (equipado/guardado/portado), datos de arma/armadura
- `spellcasting`: slots por nivel, hechizos conocidos/preparados

`by_alias=True` en `model_dump`: necesario porque el campo se llama `class` en Mongo pero `character_class` en Python.

Dónde se usa: En `routes/characters.py` para validar el body del POST/PUT. En `services/characters_service.py` en `_normalize_character_payload`.

`services/auth_service.py`

Por qué existe: Centraliza la extracción y validación del JWT para que los routes no repitan este código. FastAPI permite inyectarlo como dependencia.

`get_current_user(token) → dict`

Qué hace: Decodifica el JWT con `SECRET_KEY` y `ALGORITHM`. Verifica que tenga `user_id` y `email`. Lanza HTTP 401 si falla.

Por qué está: Patrón de dependencia de FastAPI. Cualquier route que lo incluya como `Depends(get_current_user)` queda automáticamente protegido.

Dónde se llama: En todos los endpoints que requieren auth: `PUT /monsters`, `DELETE /monsters`, `POST/PUT/DELETE /spells`, `PUT/DELETE /backgrounds`, `PUT/DELETE /items`, `GET/POST/PUT/DELETE /characters`. Es la forma estándar de proteger rutas en FastAPI.

`optional_get_current_user(request) → Optional[dict]`

Qué hace: Igual que `get_current_user` pero si no hay token o es inválido devuelve `None` en vez de lanzar 401.

Por qué está: Los endpoints `POST /monsters`, `POST /spells`, `POST /backgrounds`, `POST /items` son públicos (cualquiera puede crear) pero si hay JWT se aprovecha para setear `created_by` automáticamente. Sin este helper habría que elegir entre proteger el endpoint (y perder la creación pública) o no saber quién creó qué.

Dónde se llama: En `POST /monsters`, `POST /spells`, `POST /backgrounds`, `POST /items` como `Depends(optional_get_current_user)`.

`services/authorization_service.py`

`require_write_authorization(authorization) → dict`

Qué hace: Lee el header `Authorization: Bearer <token>` manualmente y valida el JWT.

Por qué está: Alternativa a `get_current_user` para cuando se quiere leer el header directamente en vez de usar `OAuth2PasswordBearer`. Actualmente no se usa en ninguna ruta activa — es código legado.

Dónde se llama: No está siendo usado actualmente en ningún route. `get_current_user` hace lo mismo de forma más limpia con `oauth2_scheme`.

`services/login_service.py`

Por qué existe: Separa la lógica de autenticación del handler HTTP. El route solo delega; la lógica de verificar contraseña, crear token y construir la respuesta está aquí.

`_verify_password(plain, stored) → bool`

Qué hace: Compara la contraseña en texto plano con el hash bcrypt. Si el hash no empieza por `$2` (formato bcrypt), hace comparación directa como fallback para registros legacy.

Por qué está: bcrypt genera hashes distintos cada vez con sal diferente. No se puede comparar directamente — hay que usar `checkpw`. El fallback legacy existe porque algunos datos de seed se importaron sin hashear.

Dónde se llama: Dentro de `authenticate_login` y `authenticate_login_form`, nunca desde fuera del servicio.

`_create_access_token(data) → str`

Qué hace: Crea el JWT con `jose.jwt.encode`. El payload incluye `user_id`, `email`, `username`, `isAdmin`. NO añade `exp` (sin expiración).

Por qué no expira: Decisión de diseño para simplificar el TFG. En producción real se añadiría `exp` y refresh tokens.

Dónde se llama: Dentro de `authenticate_login` y `authenticate_login_form`.

`_build_login_response(user, token) → LoginResponse`

Qué hace: Construye el objeto `LoginResponse` con el token y los datos del usuario.

Por qué está: Evitar duplicar la construcción del response en `authenticate_login` y `authenticate_login_form` que tienen el mismo resultado.

Dónde se llama: En ambas funciones de autenticación.

`authenticate_login(request) → LoginResponse`

Qué hace: Flujo completo: busca usuario por email/username, verifica contraseña, genera token.

Por qué está: Es el caso de uso principal de login desde el frontend Angular.

Dónde se llama: Desde `POST /auth/login` en `routes/login.py`.

`authenticate_login_form(form_data) → LoginResponse`

Qué hace: Igual que `authenticate_login` pero acepta `OAuth2PasswordRequestForm` (formato `x-www-form-urlencoded`).

Por qué está: Swagger UI usa este formato para el botón "Authorize". Sin este endpoint, no se podría probar la API desde la documentación interactiva.

Dónde se llama: Desde `POST /auth/token` en `routes/login.py`.

`logout_service() → dict`

Qué hace: Devuelve `{"message": "Logout correcto"}`.

Por qué está: El logout real ocurre en el cliente (borra el token de localStorage). El endpoint existe para tener un endpoint oficial de logout y porque el route requiere auth (confirma que el token era válido antes de "cerrar sesión").

Dónde se llama: Desde `POST /auth/logout` en `routes/login.py`.

`services/users_service.py`

Por qué existe: Encapsula todo el acceso a la colección `users`. Los routes no tocan MongoDB directamente.

`_to_schema(doc, include_password) → UserSchema`

Qué hace: Convierte un documento MongoDB (con `_id` ObjectId) a `UserSchema` (con `id` string). Excluye `password` salvo que `include_password=True`.

Por qué está: MongoDB devuelve `_id`, la API expone `id`. Sin esta transformación habría que repetir la conversión en cada función.

Dónde se llama: En `get_all`, `get_by_id`, `get_by_email`, `get_by_login`, `create`, `update`. Solo uso interno del service.

`get_all() → list[UserSchema]`

Dónde se llama: Desde `GET /users`. Usado en administración.

`get_by_id(user_id) → UserSchema`

Por qué está: Búsqueda por ObjectId para el CRUD de usuario individual.

Dónde se llama: Desde `GET /users/{id}`.

`get_by_login(identifier, include_password) → UserSchema | None`

Qué hace: Busca por email O username con `$or`. Con `include_password=True` incluye el hash.

Por qué está: El login acepta email o username. La query `$or` en una sola operación es más eficiente que dos queries.

Dónde se llama: Desde `authenticate_login` y `authenticate_login_form` en `login_service.py`.

`create(user_schema) → UserSchema`

Qué hace: Verifica que el email no exista. Hashea la contraseña con `bcrypt.hashpw`. Setea `created_at` y `updated_at`.

Por qué hashear aquí: La responsabilidad del hasheo está en el service, no en el route. Si el route cambia (REST → GraphQL), el hasheo sigue ocurriendo.

Dónde se llama: Desde `POST /users`. El frontend lo llama en el registro.

`update(user_id, user) → UserSchema`

Qué hace: Actualiza campos del usuario. Excluye `id`, `created_by`, `created_at` para que no puedan sobrescribirse. Actualiza `updated_at`.

Dónde se llama: Desde `PUT /users/{id}`.

`delete(user_id) → dict`

Dónde se llama: Desde `DELETE /users/{id}`.

`services/monsters_service.py`

Por qué existe: Separa la lógica de acceso a la colección `monsters` de los routes HTTP.

`_json_safe(value)`

Qué hace: Convierte recursivamente `ObjectId` a string en cualquier estructura anidada.

Por qué está: JSON no puede serializar `ObjectId`. Antes de devolver cualquier documento al cliente hay que convertirlos.

Dónde se llama: En `_format_monster` como fallback cuando la validación Pydantic falla, y en `update`.

`_build_index(name) → str`

Qué hace: Genera slug kebab-case del nombre ("Adult Gold Dragon" → "adult-gold-dragon").

Por qué está: Los monstruos D&D usan slugs como identificadores (`index`). Al crear uno nuevo sin `index` explícito, se genera desde el nombre.

Dónde se llama: En `create` cuando el payload no trae `index`.

`_format_monster(monster_data) → dict`

Qué hace: Intenta validar con `Monster.model_validate()` y hacer `model_dump`. Si falla, usa `_json_safe` como fallback.

Por qué está: Los monstruos importados de la API D&D 5e pueden tener estructuras que no encajan perfectamente en el schema. El fallback garantiza que siempre se devuelve algo.

Dónde se llama: En `get_all` y `get_by_id` antes de enviar la respuesta.

`get_all() → list`

Dónde se llama: Desde `GET /monsters/`. El componente `Monsters` lo llama en `ngOnInit` via `MonstersService.getMonsters()`.

`get_by_id(monster_id) → dict`

Qué hace: Busca por `index` (slug). Devuelve `{}` si no existe.

Por qué busca por index: El frontend navega a `/monsters/{index}` (el slug D&D), no por `ObjectId`.

Dónde se llama: Desde `GET /monsters/{id}`. La página `MonsterDetail` lo llama.

`create(monster) → dict`

Qué hace: Genera `index` si falta, valida con `Monster.model_validate`, inserta en MongoDB.

Por qué valida con Pydantic antes de insertar: Garantiza que lo que entra en la BD tiene la estructura correcta aunque el frontend envíe datos malformados.

Dónde se llama: Desde `POST /monsters`. El `MonsterForm` lo invoca al guardar.

`update(monster_id, monster) → dict`

Qué hace: Valida con `Monster.model_validate` y actualiza el documento por `index`.

Por qué busca por index en update: El frontend pasa el slug D&D del monstruo en la URL de edición.

Dónde se llama: Desde `PUT /monsters/{id}`. El `MonsterForm` en modo edición.

`delete(monster_id) → bool`

Dónde se llama: Desde `DELETE /monsters/{id}`. El componente `Monsters` al confirmar la eliminación.

`services/spells_service.py`

Por qué existe: Los hechizos tienen un problema histórico: algunos fueron importados sin campo `index`. Este service tiene lógica especial para manejar esos casos.

`_ensure_persisted_index(collection, spell_data)`

Qué hace: Si un documento no tiene `index`, lo genera desde el ObjectId y lo persiste en MongoDB.

Por qué está: Los hechizos legacy importados de la API D&D 5e no tenían `index`. Sin esto, no habría forma de buscarlos por un identificador consistente.

Dónde se llama: En `get_local_docs` (al listar todos) y `get_local_doc_by_id` (al buscar uno). Se llama en cada hechizo sin `index` para "repararlo" en caliente.

`_find_legacy_spell_by_slug(collection, spell_id)`

Qué hace: Para hechizos sin `index`, intenta encontrarlos generando el slug del nombre y comparándolo con el `spell_id` buscado.

Por qué está: Un hechizo importado como "Acid Arrow" no tiene `index`, pero alguien podría buscar por "acid-arrow". Este método cierra esa brecha.

Dónde se llama: Como último recurso en `get_local_doc_by_id` cuando la búsqueda por ObjectId e index falla.

`_sanitize_spell_input(data) → dict`

Qué hace: Limpia campos opcionales vacíos (`dc`, `damage`, `area_of_effect`, `image`) antes de validar con Pydantic.

Por qué está: El formulario de hechizos envía `{}` para campos no rellenados (ej: `dc: {}`). Pydantic rechaza `{}` como `DCSave` porque le falta `dc_type`. Este método elimina esos objetos vacíos antes de que lleguen al validador.

Dónde se llama: Al inicio de `update`, antes de `Spell.model_validate`.

`create(spell) → dict`

Dónde se llama: Desde `POST /spells`. El `SpellForm` lo invoca al guardar.

`update(spell_id, spell) → dict`

Por qué busca por ObjectId: Al editar un hechizo, el frontend usa el `id` (ObjectId) en la URL, no el slug. El ObjectId es el identificador más fiable para updates.

Dónde se llama: Desde `PUT /spells/{id}`. El `SpellForm` en modo edición.

`delete(spell_id) → bool`

Por qué solo acepta ObjectId: La eliminación es irreversible. Forzar ObjectId evita eliminar por accidente con un slug ambiguo.

Dónde se llama: Desde `DELETE /spells/{id}`. El componente `Spells` al confirmar eliminación.

`services/backgrounds_service.py`

Por qué existe: CRUD de trasfondos con lógica de timestamps y campo `meta`.

`create(background_schema) → BackgroundSchema`

Qué hace: Setea `created_by`, `created_at`, `updated_at` tanto en el documento principal como en el objeto `meta`.

Por qué duplica en meta: El campo `meta` viene de la estructura original de la API D&D 5e y se mantiene por compatibilidad.

Dónde se llama: Desde `POST /backgrounds`. El `BackgroundForm` al crear un trasfondo.

`update(background_id, background) → BackgroundSchema`

Qué hace: Excluye `id`, `created_by`, `created_at` del update (campos de solo creación). Actualiza `updated_at`.

Por qué excluye esos campos: Un update nunca debe cambiar quién creó el documento ni cuándo.

Dónde se llama: Desde `PUT /backgrounds/{id}`. El `BackgroundForm` en modo edición.

`delete(background_id) → dict`

Dónde se llama: Desde `DELETE /backgrounds/{id}`. El componente `Backgrounds`.

`services/items_service.py`

Por qué existe: Los ítems tienen 6 tipos (`weapon`, `armor`, `tool`, `mount`, `adventuringgear`, `magicitem`), cada uno con su propio schema y service. Este servicio hace de dispatcher que delega al service correcto según el tipo.

`normalize_category(category) → str | None`

Qué hace: Normaliza alias de categorías (ej: `"adventuringgear" → "adventuring-gear"`, `"magic-item" → "magic-items"`).

Por qué está: El frontend, la API D&D 5e y la BD usan variantes distintas del mismo nombre. Esta función unifica.

Dónde se llama: En `get_local_docs` para filtrar la query Mongo por categoría.

`_is_magic_item_doc(item_data) → bool`

Qué hace: Detecta si un documento es un Magic Item analizando si tiene `rarity.name`, `variant` booleano o URL con `/magic-items/`.

Por qué está: Los magic items no tienen una `equipment_category` consistente — algunos tienen "magic-items", otros no. Esta heurística multi-señal es más robusta.

Dónde se llama: En `_schema_for_doc` antes de elegir con qué schema validar.

`_schema_for_doc(item_data)`

Qué hace: Detecta el schema Pydantic correcto para un documento dado su `equipment_category` y la heurística de magic item.

Por qué está: Sin esto, al listar todos los ítems no sabríamos qué schema aplicar a cada uno. Es el "inferidor de tipo" para documentos sin tipo explícito.

Dónde se llama: En `_format_item_doc` al serializar para la respuesta.

`_normalize_type(type_value) → str | None`

Qué hace: Normaliza el tipo que viene como query param (`?type=weapon`) a un valor canónico.

Por qué está: El frontend puede enviar "weapon", "weapons", la BD puede tener "weapon". Esta función unifica antes de hacer el dispatch.

Dónde se llama: En `get_by_type`, `get_by_id`, `create`, `update`.

`get_all() → list`

Dónde se llama: Desde `GET /items`. El componente `Items` lo llama para la lista completa.

`get_by_type(type) → list`

Dónde se llama: Desde `GET /items/type/{type}`. Usado cuando se quiere listar solo armas, solo armaduras, etc.

`create(item_data, type, created_by) → dict`

Qué hace: Valida con el schema del tipo y delega al service específico.

Dónde se llama: Desde `POST /items?type=weapon`. Los form de ítems al guardar.

`delete(item_id) → dict`

Qué hace: Elimina directamente de la colección `items` por `ObjectId`.

Por qué no delega a un sub-service: La eliminación es tipo-agnóstica — solo necesita el `ObjectId`, no importa si es arma o armadura.

Dónde se llama: Desde `DELETE /items/{id}`. El componente `Items` al confirmar eliminación.

`services/characters_service.py`

Por qué existe: El personaje es el documento más complejo. Requiere validación cruzada (la clase existe en MongoDB, la raza existe, etc.) y normalización especial.

`_slugify(value) → str`

Qué hace: Genera slug del nombre del personaje para el campo `index`.

Por qué está: Todos los documentos D&D tienen `index` como identificador string. Los personajes creados por usuarios también necesitan uno.

Dónde se llama: En `_normalize_character_payload` si el payload no trae `index`.

`_resolve_reference(reference_data, collection_name) → dict`

Qué hace: Dado un objeto `{index, name, url}`, lo busca en la colección correspondiente de MongoDB y devuelve los datos actualizados.

Por qué está: Al crear un personaje, el frontend envía `{character_class: {index: "barbarian", name: "Barbarian"}}`. Este método verifica que "barbarian" realmente existe en `classes` y devuelve sus datos canónicos.

Dónde se llama: En `_normalize_character_payload` para clase, raza, subclase, subrace.

`_resolve_background(reference_data) → dict`

Por qué está separado de `_resolve_reference`: Los trasfondos se buscan en la colección `backgrounds` por `id` (ObjectId), no por `index`. La lógica es ligeramente distinta.

Dónde se llama: En `_normalize_character_payload` para el campo `background`.

`_normalize_character_payload(character_data, created_by, strict_items) → dict`

Qué hace: La función más importante del service. Toma el payload raw y lo transforma en un `CharacterSchema` válido:

1. Elimina campos gestionados internamente (`id`, `created_by`, `created_at`, `updated_at`)
2. Genera `index` desde el nombre
3. Resuelve referencias a clase, raza, trasfondo contra MongoDB
4. Valida el inventario (si `strict_items=True`, verifica que los refs de ítems existan)
5. Valida con `CharacterSchema.model_validate()`

Por qué `strict_items` es un flag: En creación (`True`) queremos verificar que los ítems existen. En actualización (`False`) los ítems ya son objetos completos embebidos, no referencias.

Dónde se llama: En `create` (con `strict_items=True`) y `update` (con `strict_items=False`). También en `_format_character_doc_async` para serializar documentos al leerlos.

`get_by_user(username) → list[dict]`

Qué hace: Obtiene personajes donde `player == username`.

Por qué filtra por `player` y no por `created_by`: El campo `player` guarda el username del creador (sacado del JWT). `created_by` es el `user_id`. El username es más legible y es lo que se guarda históricamente.

Dónde se llama: Desde `GET /characters` que requiere auth y filtra por el usuario del token.

`update_image(character_id, image_path)`

Qué hace: Actualiza solo el campo `image` y `updated_at`, sin pasar por `_normalize_character_payload`.

Por qué está separado: La normalización completa es costosa (resuelve referencias, valida toda la estructura). Para un simple cambio de imagen es innecesario y propenso a fallos si la ficha tiene datos legacy.

Dónde se llama: Desde `POST /characters/{id}/image` tras guardar el archivo de imagen.

services/class_progression_service.py

Por qué existe: La tabla de progresión de clase (Rage Count, Sneak Attack, etc.) se configura en BD para poder modificarla sin tocar el código.

get_all_progression_configs() → List[Dict]

Qué hace: Obtiene todos los configs de `class_progression_config`. Renombra `class_name` → `className`.

Por qué renombra: Python usa snake_case, el frontend Angular espera camelCase. La conversión ocurre aquí en vez de en el frontend.

Dónde se llama: Desde `GET /class-progression`. En el frontend, `ClassProgressionService.getAllProgressionConfigs()` lo usa en `class-detail.ts` para cargar el config de TODAS las clases de una vez.

get_progression_by_class(class_name) → Optional[Dict]

Dónde se llama: Desde `GET /class-progression/{class_name}`. En el frontend, `ClassProgressionService.getProgressionByClass()` lo usa en `character-detail.ts` para cargar el config de SOLO la clase del personaje activo (más eficiente que cargar todos).

create_or_update_progression(class_name, config_data)

Qué hace: Upsert: actualiza si existe, inserta si no.

Dónde se llama: Desde `POST /class-progression/{class_name}`. Para actualizar el config vía herramienta de admin o Compass.

routes/login.py — /auth

POST /auth/login

Qué hace: Recibe `LoginRequest` (email/username + password). Delega a `authenticate_login`.

Dónde se llama: Desde `LoginService.login()` en el frontend Angular.

POST /auth/token

Qué hace: Login en formato OAuth2 para Swagger UI.

Por qué está: Sin este endpoint el botón "Authorize" de `/docs` no funciona, imposibilitando probar endpoints protegidos desde la documentación.

POST /auth/logout

Qué hace: Requiere auth (valida el token). Devuelve mensaje de logout.

Por qué requiere auth: Confirma que el token era válido antes de "cerrar sesión". El borrado real del token ocurre en el cliente.

Dónde se llama: Desde `LoginService.logout()` en el header del frontend.

POST /auth/verify

Qué hace: Verifica el JWT y devuelve el payload. Respuesta: `{valid: true, user: {user_id, isAdmin, email, ...}}`.

Por qué está: El frontend no decodifica el JWT en el cliente (evita exponer la clave). En cambio, llama a este endpoint para obtener `user_id` e `isAdmin` del token almacenado.

Dónde se llama: Desde `LoginService.verifyToken()`, que es llamado por `getUserId()` y `getUserRole()`, que a su vez son llamados en el `ngOnInit` de todos los componentes lista que necesitan permisos.

routes/classes.py — /classes

GET /classes/{class_id}/levels

Qué hace: Obtiene los 20 niveles de una clase. Los niveles están embebidos en el documento de la clase en el campo `levels[]`.

Por qué embebido: Antes era una colección separada (`class_levels`) con 240 documentos que se cargaban todos en memoria y se filtraban en Python. Ahora es una única query `find_one` que devuelve el documento completo incluyendo los niveles.

Dónde se llama: Desde `ClassesService.getClassLevels()` en el frontend, que lo usa en `class-detail.ts` y `character-detail.ts` para mostrar la tabla de progresión.

GET /classes/subclasses/{id}/levels

Qué hace: Igual que el anterior pero para subclasses.

Por qué está separado del endpoint de clases: Las subclasses tienen su propia colección y sus niveles solo existen a partir del nivel en que se elige la subclase (no tienen los 20 niveles completos).

Dónde se llama: Desde `ClassesService.getSubclassFeatureProgression()`, que lo usa `class-detail.ts` al abrir el modal de una subclase.

utils/image_utils.py

parse_form_or_json(request, collection) → dict

Qué hace: Si el request es JSON, retorna `request.json()`. Si es multipart, parsea todos los campos (intenta JSON-decode cada string) y si hay un campo `image` con archivo, lo guarda en `assets/images/{collection}/` y añade la ruta al dict resultante.

Por qué está: Los formularios del frontend pueden enviar JSON (sin imagen) o multipart/form-data (con imagen). Tener un único parser transparente evita duplicar esta lógica en cada route.

Dónde se llama: En `POST/PUT /monsters`, `POST/PUT /spells`, `POST/PUT /items`. Cualquier entidad que soporte imagen.

4. Frontend — Angular 21

app.routes.ts — Routing

Por qué existe: Centraliza todas las rutas en un array que Angular usa para el router. Usa lazy loading (`loadComponent`) para no cargar todos los componentes al arrancar.

Regla de orden crítica: Las rutas específicas van ANTES de las genéricas con parámetro. Ejemplo: `/characters/new` antes de `/characters/:id`. Si se invirtiera, el router capturaría "new" como un `:id` y fallaría.

canActivate: [authGuard]: Protege rutas que requieren login. Si el usuario no está logueado, redirige a `/login?redirectTo=<ruta>`. Se aplica a: `/characters`, `/characters/new`, `/characters/:id`, todos los `/edit`, todos los `/new` de formularios.

auth.guard.ts

authGuard: CanActivateFn

Qué hace: Comprueba `loginService.isLoggedIn()` (signal que lee `localStorage.token`). Si está logueado, permite la navegación. Si no, devuelve una `UrlTree` a `/login` con el parámetro `redirectTo`.

Por qué usa `UrlTree` en vez de `router.navigate`: Devolver `UrlTree` es la forma correcta en Angular para redirigir desde un guard — permite que el router maneje la transición limpiamente sin efectos secundarios.

Dónde se llama: Angular lo invoca automáticamente antes de activar cualquier ruta que tenga `canActivate: [authGuard]`. Nunca se llama manualmente.

interfaces/Character.ts

Por qué existe: TypeScript necesita los tipos para el autocompletado y el type-check en tiempo de compilación. Sin estas interfaces, todo el código de personaje sería `any`.

CharacterDraft

Extiende `Partial<Character>`. Añade campos temporales usados solo durante el wizard de creación:

Campo	Por qué existe
<code>has_subraces?</code>	El wizard necesita saber si la raza seleccionada tiene subraces para decidir si el paso 1 está completo. Race-step lo emite; character-form lo lee en <code>canProceed()</code> .
<code>proficiency_choices_complete?</code>	El wizard necesita saber si todas las proficiency choices de la clase están seleccionadas. Class-step lo emite; character-form lo lee en <code>canProceed()</code> .
<code>racial_bonuses?</code>	Los bonificadores raciales se aplican a las stats en <code>buildPayload()</code> . Se almacenan aquí para no perderlos entre pasos.
<code>class_spell_slots?</code>	Los spell slots del nivel 1 se cargan al elegir la clase y se guardan aquí para incluirlos en el payload final.

Campo	Por qué existe
<code>background_traits?</code>	Los rasgos del trasfondo se almacenan separados de los raciales para poder listarlos juntos en el summary.
<code>statModifier(value) → number: floor((value - 10) / 2)</code> . Fórmula D&D 5e estándar.	
Por qué aquí: Es la función más usada en toda la ficha de personaje. Se exporta desde el interface para que cualquier componente la importe sin crear dependencias circulares.	
Dónde se llama: En <code>character-detail.ts</code> para calcular todos los modificadores de stats, saving throws y habilidades. En <code>character-form.ts</code> para calcular el HP máximo inicial.	
<code>signedModifier(value) → string</code> : Devuelve "+3" o "-1".	
Por qué está: La UI D&D siempre muestra el modificador con signo explícito. Centralizar el formato aquí evita duplicar el ternario <code>mod >= 0 ? '+' + mod : mod</code> en cada template.	
Dónde se llama: En <code>character-detail.html</code> en docenas de sitios para mostrar modificadores.	

interfaces/class-progression-config.ts

Por qué existe: El config de progresión de clase se deserializa desde la API y se pasa entre `ClassProgressionService`, `class-detail.ts` y `character-detail.ts`. Necesita tipado para que TypeScript detecte errores.

ProgressionColumn

`{id, label, cssClass?, source?, key?, progression[]}`.

Diferencia entre `progression` y `source+key`:

- Con `progression[]`: el valor para el nivel X se busca en el array (útil para valores formateados como "+2", "d6")
- Con `source+key`: el valor se lee directamente del campo `class_specific` de la API para ese nivel (útil cuando la BD ya tiene el valor exacto)

services/login-service.ts

Por qué existe: Centraliza la autenticación en el frontend. Todos los componentes que necesitan saber si el usuario está logueado o quién es, inyectan este service.

isLoggedIn (signal)

Por qué es un signal: Los signals en Angular 21 son reactivos — cualquier template que lea `loginService.isLoggedIn()` se actualiza automáticamente cuando cambia. El header, el guard, los botones de login/logout todos reaccionan sin suscripciones manuales.

Dónde se usa: En `auth.guard.ts` para proteger rutas. En `din-header` para mostrar/ocultar el botón de login.

verifyToken(token) → Promise<any>

Qué hace: `POST /auth/verify` con el token. Devuelve el payload del JWT.

Por qué llama al backend en vez de decodificar el JWT en el cliente: Decisión de seguridad y simplicidad. No se necesita la librería de JWT en el cliente, y el backend tiene la clave para verificar la firma.

Dónde se llama: En `getId()` y `getUserRole()`. Solo uso interno del service.

`getId() → Promise<string | null>`

Qué hace: Llama a `verifyToken` y extrae `user.user_id`.

Por qué está: Cada componente lista necesita el `userId` para determinar si puede editar/eliminar un ítem (`canEdit`). En vez de llamar a `verifyToken` directamente y extraer el campo, este helper encapsula el detalle.

Dónde se llama: En `ngOnInit` de `Monsters`, `Spells`, `Backgrounds`, `Items`, `Classes`. También en `character-detail.ts`.

`getUserRole() → Promise<string | null>`

Qué hace: Llama a `verifyToken` y convierte `isAdmin: true → 'admin', false → 'user'`.

Por qué la conversión: La API devuelve un booleano `isAdmin`. El frontend usa strings como rol para poder comparar `role === 'admin'`.

Dónde se llama: En `ngOnInit` de `Monsters`, `Spells`, `Backgrounds`, `Items`. Junto con `getId` para determinar permisos.

`login(email, password)`

Qué hace: `POST /auth/login`. Si tiene éxito, guarda el token en `localStorage` y setea `isLoggedIn = true`.

Por qué localStorage: Las cookies requieren más configuración (CORS, SameSite). `localStorage` es más simple para un TFG.

Dónde se llama: En `Login.onSubmit()`.

`logout()`

Qué hace: Borra `localStorage.token`, resetea signals, navega a `/login` con `window.location.href`.

Por qué window.location.href en vez de router.navigate: `window.location.href` fuerza recarga completa, limpiando todo el estado de Angular en memoria (caché de services, signals, etc.). Sin esto, datos del usuario anterior podrían quedar en memoria.

Dónde se llama: Desde el botón de logout en `din-header`.

`services/classes-service.ts`

`_classes` (signal privado, caché)

Por qué existe: Cargar todas las clases implica: lista de previews + detalle de cada una = 13 HTTP requests (1 + 12). Con caché, la segunda visita al manual es instantánea.

Riesgo: Si se crea una clase nueva en otra sesión, la caché no se actualiza hasta que el usuario recarga. Aceptable para el TFG.

`getClasses(onItemLoaded?) → Promise<DndClass[]>`

Qué hace: Si hay caché, la devuelve y llama `onItemLoaded` para cada clase (para actualizar la UI progresivamente). Si no hay caché, carga la lista de previews y luego los detalles en paralelo con `Promise.all`.

Por qué el callback `onItemLoaded`: Las clases se cargan en paralelo. Sin callback, el usuario vería la pantalla vacía hasta que todas terminen. Con callback, cada clase aparece en cuanto termina de cargarse.

Dónde se llama: En `Classes.ngOnInit()` (lista del manual), `ClassStepComponent.ngOnInit()` (wizard de PJ), `ClassDetail.ngOnInit()`.

`getClassLevels(id) → Promise<DndClassLevel[]>`

Qué hace: `GET /classes/{id}/levels`. Los datos vienen del campo `levels[]` embebido en el documento de clase.

Por qué no cachea como `getClasses`: Los niveles son datos fijos que no cambian. Pero como se usan en contextos muy específicos (class-detail y character-detail), la ganancia de cachear no justifica la complejidad.

Dónde se llama: En `ClassDetail.ngOnInit()` para la tabla de progresión. En `ClassStepComponent.selectClass()` para obtener spell slots de nivel 1. En `character-detail.loadClassLevelData()` para la ficha de personaje. En `character-detail.confirmLevelUp()` para subir de nivel.

`getClassFeatureProgression(classId) → Promise<LeveledFeature[]>`

Qué hace: Carga los niveles, luego para cada feature referenciada carga su detalle en paralelo. Deduplica features y ordena por nivel.

Por qué deduplica: Algunas features aparecen en múltiples niveles (ej: Extra Attack en nivel 5 y en el resumen de nivel 20). Sin deduplicación aparecerían dos veces.

Dónde se llama: En `ClassDetail.ngOnInit()` para la sección "New Features". En `character-detail.loadDynamicTraits()` para añadir los rasgos de clase al personaje.

`services/class-progression-service.ts`

Por qué existe: La tabla de progresión de clase (Rage Damage, Sneak Attack, etc.) se configura en BD. Este service es el puente entre esa colección y el frontend.

`getAllProgressionConfigs() → Promise<AllClassesProgressionConfig>`

Qué hace: `GET /class-progression`. Convierte el array de la API a un mapa `{className → config}`.

Por qué convertir a mapa: `class-detail.ts` necesita acceder al config de una clase por nombre (`config['barbarian']`). El array requeriría un `find()` en cada acceso; el mapa es $O(1)$.

Dónde se llama: En `ClassDetail.ngOnInit()` para cargar los configs de TODAS las clases de una vez (ya que no se sabe qué clase se va a ver).

`getProgressionByClass(className) → Promise<ClassProgressionConfig>`

Qué hace: `GET /class-progression/{className}`. Solo carga el config de UNA clase.

Por qué no usar `getAllProgressionConfigs` aquí también: En `character-detail.ts` ya sabemos la clase del personaje. Cargar solo su config es más eficiente.

Dónde se llama: En `character-detail.loadClassLevelData()` en paralelo con `getClassLevels()`. Solo se llama una vez por personaje cargado.

`services/characters-service.ts`

`save(character)` — **debounced 800ms**

Qué hace: Espera 800ms desde el último cambio y luego hace `PUT /characters/{id}`.

Por qué debounced: El usuario puede editar campos rápidamente (tippear en un textarea, ajustar HP varias veces). Sin debounce, cada keypress haría un request HTTP. Con 800ms, solo se guarda cuando el usuario para de editar.

Dónde se llama: En casi todos los métodos de edición de `character-detail.ts`: cambio de stats, notas, bio, inventario, etc.

`saveImmediate(character)`

Qué hace: Cancela el timer del debounce y guarda inmediatamente.

Por qué existe: Si el usuario edita algo y luego navega a otro sitio antes de que pasen los 800ms, el componente se destruye con el timer pendiente y el cambio se pierde. `saveImmediate` garantiza el guardado antes de navegar.

Dónde se llama: En `character-detail.goToSheet()` antes de navegar a la hoja de personaje. Y en cualquier acción importante como subir de nivel.

`services/monsters-service.ts`

`delete(id)`

Por qué incluye el JWT manualmente: El DELETE de monstruos requiere auth. El interceptor HTTP lo añade automáticamente para todos los requests, pero este método muestra que el diseño original lo hacía explícito.

Dónde se llama: En `Monsters.deleteMonster()` tras confirmar el diálogo. El componente luego elimina el monstruo de sus signals locales sin necesidad de recargar la lista.

`components/alerts/alerts.ts`

Por qué existe: `window.alert()` es bloqueante, no tiene estilo, y no se puede personalizar. Este componente proporciona una alternativa modal con variantes (info/success/error) y estilo D&D.

`open(): void` y `close(): void`

Por qué el componente no se abre/cierra solo: Sigue el patrón de "componente tonto" — la lógica de cuándo mostrar la alerta está en el componente padre (vía `[isOpen]`), no dentro del modal. Esto facilita el testing y el reuso.

Dónde se usa: En `Monsters`, `Spells`, `Backgrounds`, `Items`. Cuando falla la operación `delete`, el componente padre setea `alertOpen.set(true)` y el modal se muestra.

`components/filter-modal/filter-modal.ts`

Por qué existe: Los filtros de tipo múltiple (ej: "filtrar por schools: Evocation, Necromancy") necesitan una UI de selección múltiple con búsqueda. Este componente es genérico y reutilizable.

`open()`

Qué hace: Copia `selectedOptions` a `localSelected`, resetea la búsqueda y abre el modal.

Por qué copia a local: Si el usuario abre el modal, hace selecciones pero cancela, no queremos que los cambios se apliquen. Al trabajar con una copia local, `cancel` simplemente cierra sin tocar el estado externo.

Dónde se llama: Desde los botones de filtro en `spells.html`, `monsters.html`, `items.html` vía `#ref.open()` con `@ViewChild`.

`confirm()`

Qué hace: Emite `localSelected` (las selecciones confirmadas) y cierra.

Por qué emit en vez de two-way binding: El componente padre decide qué hacer con las selecciones (aplicar filtros, actualizar URL). El modal no sabe de dónde viene ni qué hace el padre.

Dónde se llama: Desde el botón "Confirm" dentro del modal HTML.

`components/monsters/monsters.ts` (patrón de las 6 listas)

Las 6 listas del manual (`Races`, `Classes`, `Backgrounds`, `Spells`, `Monsters`, `Items`) comparten el mismo patrón.

`getSourceLabel(item) → 'official' | 'community' | 'own'`

Qué hace: Clasifica un ítem según su `created_by`:

- Sin `created_by` o `=== 'oficial' → 'official'`
- `created_by === userId() → 'own'`
- Cualquier otro `→ 'community'`

Por qué está en cada componente en vez de un service: Es una función pura de 3 líneas. Extraerla a un service añadiría complejidad sin beneficio. Si necesita cambiar, está en 5 sitios bien localizados.

Dónde se llama: En `applyFilters()` cuando `filters.source !== 'all'`.

`canEdit(item) → boolean`

Qué hace: Devuelve `true` si el usuario es admin O si `item.created_by === userId()`.

Por qué está en el componente: La lógica de permisos es UI — decide si mostrar los botones Edit/Delete. No tiene sentido en un service de datos.

Dónde se llama: En el template `@if (permissionsLoaded() && canEdit(item))` para envolver los botones de acción.

`showAlert(title, message, variant)`

Qué hace: Setea los signals del modal `Alerts` y lo abre.

Por qué está aquí: Cada componente tiene su propia instancia del modal `<app-alerts>`. Esta función es el

helper que evita repetir los 4 `.set()` en cada catch.

Dónde se llama: En el `catch` de `deleteMonster/deleteSpell/deleteBackground/deleteItem`.

`navigateToCreate()` y `navigateToEdit(item)`

Qué hace: Navega al formulario de creación/edición pasando `returnUrl = window.location.pathname + window.location.search`.

Por qué `returnUrl`: Al salir del formulario (submit o cancel), el form navega a `window.location.href = this.returnUrl` en vez de una URL hardcodeda. Esto restaura la posición exacta de la lista con todos los filtros aplicados.

Dónde se llama: Desde los botones "+ Create" y "Edit" en el template.

`applyFilters()`

Por qué resetea `currentPage` a 1: Si el usuario está en la página 3 y aplica un filtro que reduce los resultados a 5 ítems, la página 3 puede no existir. Resetear a 1 evita quedarse en una página vacía.

Dónde se llama: En `onFilterChange()` (cualquier cambio de filtro) y en `loadXxx()` tras recibir todos los datos del backend.

`pages/class-detail/class-detail.ts`

`extractDisplayColumns(levels, config, className) → DisplayColumn[]`

Qué hace: Construye la lista de columnas para la tabla de progresión de clase:

1. Añade columnas del config de BD con sus labels y valores del array `progression[]`
2. Auto-detecta keys de `class_specific` NO cubiertas por el config (para clases sin config o con datos extra)
3. Añade columnas de spell slots si `config.spellSlots === true`

Por qué auto-detecta: Si un día se añade un nuevo campo `class_specific` en la BD sin actualizar el config, la columna aparece automáticamente con un label generado. Sin esto habría que actualizar el config manualmente.

coveredKeys: Set de `col.key` de las columnas ya configuradas. Se usa para evitar añadir dos veces una key que ya tiene columna (ej: `rage_count` configurado como "Rages per Day" no debe aparecer también como "Rage Count" desde el auto-detect).

Por qué compara por key y no por id: Los IDs configurados ('`rages-count`') son distintos de los auto-generados ('`class_specific_rage_count`'). La deduplicación debe ser por el campo de datos que ambas referencias, no por el ID de columna.

Dónde se llama: En `ngOnInit` tras cargar clase, niveles y config. Solo se llama una vez.

`getColumnValue(level, column) → string | number`

Qué hace: Para una columna y un nivel concreto, devuelve el valor a mostrar.

- Si la columna tiene `progression[]`: recorre el array tomando el último entry cuyo `level ≤ level.level`

- Si tiene `source + key`: lee directamente del objeto `class_specific` o `spellcasting`
- Si tiene ambos: `progression[]` tiene prioridad (el config manual manda)

Por qué toma el último entry (no el exacto): La tabla D&D no repite un valor si no cambia. Si Rage Damage es "+2" desde nivel 1 hasta nivel 8, el array tiene solo `{level: 1, value: "+2"}`. Para niveles 2-8 hay que tomar el último entry anterior.

Dónde se llama: En el template para cada celda de la tabla `@for (level of levels()) ... @for (column of displayColumns())`.

`scrollToFeature(index)`

Qué hace: Hace scroll suave hasta el elemento con id `feature-{{index}}`.

Por qué está: La tabla de progresión muestra los nombres de las features con el nivel. Al hacer click en un nombre se salta a su descripción en la sección inferior de la página.

Dónde se llama: Desde `(click)="scrollToFeature(feature.index)"` en la tabla de niveles del template.

`pages/character-detail/character-detail.ts`

El componente más complejo del frontend. Gestiona la ficha completa de personaje de forma interactiva y persistente.

`loadClassLevelData(c)`

Qué hace: Lanza en paralelo dos requests:

1. `classesService.getClassLevels(classId)` — niveles de la clase
2. `classProgressionService.getProgressionByClass(classId)` — config de columnas

Por qué en paralelo: Ambos datos son independientes. En paralelo se obtienen en el tiempo del más lento (no la suma de ambos).

Por qué `.catch(() => null)` en el config: Si el config no existe para la clase (clases custom o futura), no debe romper la carga del personaje. Con `null` el computed `classSpecificEntries` cae al path B (raw `class_specific`).

Dónde se llama: En `ngOnInit` tras cargar el personaje, de forma non-blocking (sin `await` en el stream principal). La ficha se muestra aunque los datos de progresión aún estén cargando.

`classSpecificEntries (computed)`

Qué hace: Calcula las entradas de la sección "Class Features" de la ficha:

Path A (con config): Itera `progressionColumns`, evalúa `progression[]` para el nivel del personaje, filtra los valores '-' (features no desbloqueadas aún).

Path B (fallback sin config): Lee raw `class_specific` del nivel actual, auto-genera labels, filtra ceros y nulos.

Por qué computed y no método: Un computed en Angular 21 solo se recalcula cuando cambian sus dependencias (`classProgressionConfig()`, `classLevelData()`, `character()`). Un método se recalcularía en cada ciclo de detección de cambios.

Dónde se usa: Directamente en el template en la sección "Class Features" con `@for (entry of classSpecificEntries())`.

`skillModifier(skill) → number`

Qué hace: Calcula el modificador total de una habilidad: modificador de stat + proficiency si corresponde + doble proficiency si tiene expertise + bonus extra si existe.

Por qué está aquí: Es una fórmula D&D 5e pura que no tiene estado. Podría estar en el interface pero al necesitar acceder al personaje, tiene sentido en el componente.

Dónde se llama: En el template para cada skill en la pestaña "basic", y en `character-sheet.ts` para la hoja imprimible.

`confirmLevelUp()`

Qué hace:

1. Incrementa `level` en 1
2. Obtiene spell slots del nuevo nivel y los actualiza
3. Carga features del nuevo nivel y las añade a `traits`
4. Si el nivel tiene ASI, muestra el selector de stat

Por qué está en el componente: El level-up es una transacción compleja que afecta varios campos del personaje (level, spell_slots, traits). Necesita acceso al state del personaje y a los services. Un service de level-up sería overkill para este caso.

Dónde se llama: Desde el botón "Level Up" en el template, que solo es visible cuando `character.level < 20`.

`saveImmediate()` y `save()` (debounced)

Dónde se llaman: `save()` en todos los cambios pequeños (editar bio, ajustar stats, marcar/desmarcar slots). `saveImmediate()` en acciones importantes antes de navegar: `goToSheet()`, `confirmLevelUp()`.

`pages/character-form/character-form.ts`

Por qué existe: El wizard de creación de personaje necesita un orquestador que:

1. Mantiene el `draft` acumulando los cambios de cada paso
2. Valida si el usuario puede avanzar al siguiente paso
3. Construye el payload final y llama al backend

`canProceed() → boolean`

Qué hace: Valida el estado actual del draft según el paso:

Paso	Condición	Por qué
1 (Race)	<code>!!d.race && (!d.has_subraces !!d.subrace)</code>	Si la raza tiene subraces, elegirla es obligatorio (define rasgos y bonificadores)
2 (Class)	<code>!!d.character_class && (d.proficiency_choices_complete !==</code>	Las proficiencias de clase definen qué skills tiene el personaje

Paso	Condición	Por qué
	<code>false</code>)	
3 (Background)	<code>!!d.background</code>	El trasfondo es obligatorio para los rasgos de RP
4 (Stats)	6 stats asignados	Sin stats no se puede calcular el HP ni los modificadores
5 (Summary)	<code>!!d.name?.trim()</code>	El personaje debe tener nombre

`d.proficiency_choices_complete !== false` (no `=== true`): Si el campo es `undefined` (no se ha emitido aún), pasa la validación para no bloquear el avance. Solo bloquea si explícitamente es `false`.

Dónde se llama: Desde el botón "Next" del template (`[disabled]="!canProceed()"`) y al inicio de `nextStep()`.

`buildPayload()`

Qué hace: Construye el objeto a enviar al backend aplicando bonificadores raciales a las estadísticas base del personaje.

Por qué los bonificadores se aplican aquí: El draft almacena los stats base (los que el usuario elige en el paso 4) y los bonificadores raciales por separado. El personaje final tiene los stats con los bonificadores ya aplicados.

Dónde se llama: Solo en `submit()`.

`pages/character-form/steps/race-step/race-step.ts`

`selectRace(race, emit)`

Qué hace: Limpia la subrace, espera a que las subraces carguen del API, y DESPUÉS emite el cambio al padre con `has_subraces = this.subraces().length > 0`.

Por qué emite después de cargar: Si emitiera antes, `has_subraces` podría ser `false` (subraces aún no cargadas) aunque la raza sí tenga subraces. El wizard avanzaría sin exigir la subrace.

Dónde se llama: Al hacer click en una tarjeta de raza en el template, y en `ngOnInit` para restaurar la raza del draft si se vuelve al paso.

`selectSubrace(subrace)`

Qué hace: Toggle — si ya estaba seleccionada la deselecciona.

Por qué toggle: Permite al usuario "arrepentirse" de elegir una subrace y quedar sin subrace seleccionada (lo cual bloquea el avance en `canProceed()`).

Dónde se llama: Al hacer click en una tarjeta de subrace.

`emitChange(race, subrace)`

Qué hace: Construye el patch del draft con: referencia a raza, referencia a subrace, velocidad, bonificadores raciales, rasgos de raza, y el flag `has_subraces`.

Por qué incluye bonificadores raciales: El paso de stats necesita saber los bonificadores para mostrar el total

final al usuario. Se calculan aquí porque es donde se tienen los datos de la raza.

Dónde se llama: Desde `selectRace` y `selectSubrace`. Siempre emite al padre para mantener el draft sincronizado.

pages/character-form/steps/class-step/class-step.ts

`selectClass(cls)`

Qué hace: Resuelve las proficiency choices, resetea selecciones, carga spell slots de nivel 1, emite cambio.

Por qué carga spell slots de nivel 1 aquí: El personaje empieza en nivel 1. Los slots de nivel 1 son la "semilla" del sistema de magia del personaje. Si se seleccionan en el wizard, ya están disponibles desde el primer momento en la ficha.

Dónde se llama: Al hacer click en una tarjeta de clase.

`toggleChoice(groupIndex, optionIndex)`

Qué hace: Añade/quita una proficiencia del grupo. Respeta el máximo `choose`. Emite cambio con el nuevo `proficiency_choices_complete`.

proficiency_choices_complete: `resolved.every((choice, idx) => byGroup[idx].length >= choice.choose)`. Para clases sin choices (`resolved` vacío), `every` sobre array vacío devuelve `true` (vacuamente verdadero).

Dónde se llama: Al hacer click en un botón de proficiencia en el template.

`resolveChoices(choices) → ResolvedChoice[]`

Qué hace: Transforma el formato de la API D&D 5e (`from.options[].item`) al formato interno `{desc, choose, options[]}`.

Por qué existe: El formato de la API es verbose y anidado. El formato interno es más limpio para renderizar en el template.

Dónde se llama: Solo en `selectClass` al cargar una nueva clase.

5. Flujos End-to-End

Flujo: Login y obtención de permisos

1. Usuario entra `/login` → `LoginComponent.onSubmit()`
2. `loginService.login(email, password)`
3. `POST /auth/login` → `login_service.authenticate_login()`
4. `_verify_password()` verifica bcrypt hash
5. `_create_access_token()` crea JWT con `{user_id, email, isAdmin}`
6. `localStorage.token = access_token`
7. `loginService.isLoggedIn.set(true)`
8. Router navega a `/` (o al `redirectTo`)
- En cualquier lista ---
9. `Monsters.ngOnInit()`
10. `getUserId()` → `POST /auth/verify` → extrae `user_id` del payload

11. `getUserRole()` → `POST /auth/verify` → convierte `isAdmin` a `'admin'|'user'`
12. `permissionsLoaded.set(true)`
13. Template renderiza botones Edit/Delete para ítems donde `canEdit() = true`

Flujo: Creación de contenido con auto-set de `created_by`

1. Usuario (logueado) hace click en "+ Create Monster"
2. `Monsters.navigateToCreate()`
 - captura `returnUrl = window.location.pathname + window.location.search`
 - `router.navigate(['/monsters/new'], { queryParams: { returnUrl } })`
3. `MonsterForm.ngOnInit()` lee `returnUrl`
4. Usuario rellena el form y envía
5. `MonsterForm.onSubmit()`
 - `buildFormData()` serializa a `FormData` (si hay imagen) o `JSON`
 - `monstersService.create(data)`
 - `POST /monsters` (con `Authorization: Bearer {token}`)
6. `monsters.py: create_monster(request, current_user = Depends(optional_get_current_user))`
 - `optional_get_current_user()` extrae `user_id` del JWT sin lanzar 401
 - `monster_data["created_by"] = user_id`
 - `monsters_service.create(monster_data)`
7. MongoDB: inserta `{name: "...", created_by: "abc123", ...}`
8. `MonsterForm: window.location.href = returnUrl`
 - Browser recarga la lista con los mismos filtros que había

Flujo: Filtrado por source

1. Usuario selecciona "Own" en el filtro Content
2. `Monsters.onFilterChange()`
 - `filters.set({...currentFilters, source: 'own'})`
 - `currentPage.set(1)`
 - `applyFilters()`
3. `applyFilters()`:
 - `filtered = allMonsters().filter(m => getSourceLabel(m) === 'own')`
4. `getSourceLabel(m)`:
 - `if !m.created_by → 'official'`
 - `if m.created_by === 'oficial' → 'official'`
 - `if m.created_by === userId() → 'own' ← match`
 - `else → 'community'`
5. `filteredMonsters.set(filtered)`
6. `updatePaginatedMonsters() → paginatedMonsters.set(slice)`
7. Template re-renderiza con solo los monstruos del usuario

Flujo: Tabla de progresión de clase (class-detail)

1. Usuario navega a /classes/barbarian
2. ClassDetail.ngOnInit()
3. Promise.all([
 - classesService.getClass(id),
 - classesService.getClassLevels(id), ← lee classes[].levels (embebido)
 - classesService.getSubclassesByClass(id),
 - classesService.getClassFeatureProgression(id),
 - classProgressionService.getAllProgressionConfigs()
])
4. extractDisplayColumns(levelData, config, 'barbarian')
 - a. Añade columnas del config:
 - "Rage Damage" (col.key = 'rage_damage_bonus')
 - "Rages per Day" (col.key = 'rage_count')
 - b. coveredKeys = {'rage_damage_bonus', 'rage_count'}
 - c. Auto-detecta class_specific keys de los niveles:
 - 'rage_damage_bonus' → en coveredKeys → SKIP
 - 'rage_count' → en coveredKeys → SKIP
 - 'brutal_critical_dice' → NO en coveredKeys → añade "Brutal Critical Dice"
 - d. No spellSlots (barbarian no es caster)
5. displayColumns = [{Rage Damage}, {Rages per Day}, {Brutal Critical Dice}]
6. Template renderiza tabla: para cada nivel, getColumnValue(level, column)
 - "Rage Damage" nivel 1: progression[{level:1, value:'+2'}] → '+2'
 - "Rages per Day" nivel 5: progression[{1:'2'}, {3:'3'}].last(≤5) → '3'
 - "Brutal Critical Dice" nivel 1: class_specific.brutal_critical_dice = 0 → '0'

Flujo: Tabla de progresión en ficha de personaje

1. CharacterDetail.ngOnInit() carga el personaje
2. loadClassLevelData(character) – non-blocking
3. Promise.all([
 - classesService.getClassLevels(classId), ← todos los niveles
 - classProgressionService.getProgressionByClass(classId) ← solo esta clase
])
4. classLevelData.set(levels.find(l => l.level === character.level))
5. classProgressionConfig.set(config)
6. classSpecificEntries computed se recalcula:
 - charLevel = character.level (ej: 5)
 - config.progressionColumns existe → Path A
 - "Rages per Day": progression[{1:'2'}, {3:'3'}, {6:'4'}].last(≤5) = '3'
 - "Rage Damage": progression[{1:'+2'}, {9:'+3'}].last(≤5) = '+2'
 - Filtrar '-' → sin cambios
7. Template muestra: Rages per Day: 3 | Rage Damage: +2

Flujo: Wizard de creación de personaje

Paso 1 – Race:

```
RaceStep.selectRace(race)
  → await getSubracesByRace(race.index) → subraces.set(subs)
```

```
→ emitChange(race, null) con has_subraces = subraces().length > 0
→ CharacterForm.patchDraft({race, has_subraces: true, ...})
Si has_subraces: usuario DEBE elegir subrace
canProceed() = !!d.race && (!d.has_subraces || !!d.subrace)
```

Paso 2 – Class:

```
ClassStep.selectClass(cls)
→ resolveChoices() extrae opciones de proficiencias
→ selectedByGroup.set([], []) – reset selecciones
→ await getClassLevels → extrae spell slots nivel 1
→ emitChange → proficiency_choices_complete: false (no hay selecciones aún)
Usuario elige proficiencias:
toggleChoice(0, 'skill-athletics')
→ proficiency_choices_complete = every group filled → true/false
canProceed() = !!d.character_class && (d.proficiency_choices_complete !== false)
```

Pasos 3-4: background, stats – straightforward

Paso 5 – Submit:

```
CharacterForm.submit()
→ buildPayload(): stats + bonificadores raciales
→ charactersService.createCharacter(payload)
→ POST /characters (requiere auth)
→ characters_service.create() normaliza, resuelve refs, valida, inserta
→ Si hay imagen: uploadImage(id, file) → POST /characters/{id}/image
→ router.navigate(['/characters', id])
```